Министерство науки и высшего образования Российской Федерации

ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт компьютерных систем и информационной безопасности
Кафедра информационных систем и программирования
Направление подготовки 09.03.04 Программная инженерия
(код и наименование направления подготовки)
Профиль Проектирование и разработка программного обеспечения
(наименование профиля)

КУРСОВАЯ РАБОТА

по дисциплине Алгоритмы и структуры данных
(наименование дисциплины)
на тему: Исследование блочного умножения матриц
(тема курсовой работы)

Выполнила студентка Лисненко Мария Максимовна курса 1 группы 24-КБ-ПР2
(фамилия, имя, отчество)
Допущена к защите 21.03.2025
(дата)

Руководитель (нормоконтролёр) работы проф. _____ Марков В.Н.
(должность, подпись, дата)

Защищена 21.03.2025 Оценка Отлично.
(дата)

Члены комиссии: _____ канд. техн. наук, доцент Мурлина В.А.
_____ ст.преп. Рудник Н.Т.
(должность, подпись, дата, расшифровка подписи)

Краснодар
2025 г

ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт компьютерных систем и информационной безопасности
Кафедра информационных систем и программирования
Направление подготовки/специальность 09.03.04 Программная инженерия
(код и наименование направления подготовки/специальности)

Профиль/специализация Проектирование и разработка программного обеспечения
(наименование профиля/специализации)

УТВЕРЖДАЮ
Зав. кафедрой доц. Янаева М.В.
_____
« 13 » февраля 2025 г.

ЗАДАНИЕ
на курсовую работу

Студентке Лисненко Марии Максимовне курса 1 группы 24-КБ-ПР2
Тема работы: «Исследование блочного умножения матриц»
(утверждена указанием директора института № 29-Кт от 13.02.2025 г.)

План работы:
1. Разработать генератор двумерных массивов заданной размерности, кратной размеру блоков.
2. Разработать программу блочного умножения матриц.
3. Экспериментально определить функциональную зависимость времени умножения квадратных матриц на себя от их размера. Сравнить со временем традиционного умножения матриц.

Объем работы:
а) пояснительная записка 20 с.
б) иллюстративная часть 4 листа.

Рекомендуемая литература:
1. Белов В.В., Чистякова В.И. Алгоритмы и структуры данных [Электронный ресурс]: Учебник / В.В. Белов, В.И. Чистякова. – Москва: КУРС: ИНФРА-М: 2020. – 240 с. Url: https://znanium.com/bookread2.php?book=1057212.
2. Колдаев В.Д. Структуры и алгоритмы обработки данных [Электронный ресурс]: Учеб. пособие. – М.: РИОР: ИНФРА-М, 2014. – 296 с. Url: https://znanium.com/bookread2.php?book=418290.
3. Марков В.Н. Алгоритмы и структуры данных: учеб. пособие/ В.Н. Марков. – Краснодар: Изд. ФГБОУ ВО «КубГТУ», 2022. – 207 с.

Срок выполнения: с « 15 » февраля по « 07 » июня 2025 г.

Срок защиты: « 07 » июня 2025 г.

Дата выдачи задания: « 15 » февраля 2025 г.

Дата сдачи работы на кафедру: « 07 » июня 2025 г.

Руководитель работы профессор кафедры _____ Марков В.Н.
(должность, подпись)

Задание приняла студентка _____ Лисненко М.М.
(подпись)

Реферат

Пояснительная записка курсовой работы содержит: 16 с., 3 рис., 2 табл., 4 источника, 1 приложения.

БЛОЧНОЕ УМНОЖЕНИЕ МАТРИЦ, АЛГОРИТМЫ УМНОЖЕНИЯ, ОПТИМИЗАЦИЯ АЛГОРИТМОВ

Целью данной курсовой работы является определение функциональной зависимости времени умножения матриц на себя от размера блока.

Объектом исследования является не только сам процесс блочного умножения матриц, но и его практическое применение и эффективность в современных вычислительных системах.

Курсовая работа содержит теоретическую часть, практическую часть, полученные результаты. Данная работа также включает в себя программный код, графики и таблицы, сравнительный анализ и заключение (обобщение основных результатов исследования, выводы по поставленным задачам).

Программное обеспечение создано в среде Microsoft Visual Studio Community 2022 на языке программирования C#. Иллюстрации выполнены в Microsoft Excel 2018.

Содержание

Введение 5

1. Разработка алгоритма 6

1.1. Постановка задач 6

1.2. Разработка генератора двумерных массивов 6

1.3. Программная реализация 7

1.4. Описание алгоритма 9

1.5. Вывод программы 11

2. Определение функциональной зависимости 12

Заключение 13

Список использованных источников 14

Приложение. Проверка на антиплагиат 15

Введение

В современном мире вычислительных технологий матричные операции играют ключевую роль в различных областях, таких как компьютерная графика, машинное обучение, обработка сигналов и решение систем линейных уравнений. Одной из наиболее важных и часто используемых операций является умножение матриц. Однако с увеличением размеров матриц и объемов данных, традиционные алгоритмы умножения становятся недостаточно эффективными. В связи с этим, блочное умножение матриц представляет собой перспективный подход, который позволяет значительно оптимизировать процесс вычислений. Данный метод основывается на разбиении больших матриц на более мелкие блоки, что способствует более эффективному использованию кэш-памяти и параллельной обработке данных.

Целью данной курсовой работы является определение функциональной зависимости времени умножения матриц на себя от размера блока.

Для достижения поставленной цели были поставлены следующие задачи:

1. Разработать генератор двумерных массивов заданной размерности, кратной размеру блоков.
2. Разработать программу блочного умножения матриц.
3. Экспериментально определить функциональную зависимость времени умножения матриц на себя от размера блока. Найти оптимальный размер блока.

Таким образом, данное исследование не только углубляет понимание алгоритмов умножения матриц, но и открывает новые горизонты для оптимизации вычислительных процессов в различных научных и инженерных приложениях.

1. Разработка алгоритма

1.1. Постановка задач

Анализ теоретических основ блочного умножения матриц. Разработка алгоритма блочного умножения. Разработать генератор двумерных массивов заданной размерности, кратной размеру блоков.

Далее разработать программу блочного умножения матриц. Применение результатов в реальных задачах.

1.2. Разработка генератора двумерных массивов

Для адекватной оценки эффективности алгоритма блочного умножения матриц необходимо разработать надёжный и высокопроизводительный генератор двумерных массивов. Данный генератор должен быть способен создавать матрицы заданных размеров, соответствующих требованиям, необходимым для проведения экспериментальных вычислений.

1. Совпадение размерностей: Размерности генерируемых матриц должны быть кратны размеру выбранного блока. Это условие критически важно для корректного выполнения блочного умножения, так как оно обеспечивает возможность деления матриц на подматрицы, что является основой данного метода.

2. Разнообразие элементов: Элементы массивов должны заполняться числами, соответствующими заранее определённым критериям. Это необходимо для обеспечения разнообразия входных данных и исключения специфических случаев, которые могут исказить результаты анализа производительности. Например, использование случайных чисел в заданном диапазоне или применение заранее заданных распределений позволит создать более универсальные тестовые наборы.

Кроме того, важно гарантировать, что подготовленные данные охватывают широкий спектр возможных входных параметров. Это исключит уз-

кие или специфические случаи, которые могут существенно повлиять на результаты тестирования алгоритма.

Каждое из этих условий играет ключевую роль в процессе разработки и направлено на создание оптимальной среды для испытания алгоритма блочного умножения матриц. Такой подход обеспечит достоверные и воспроизводимые результаты.

1.3. Программная реализация

Листинг

```
using System;
class BlockMatrixMultiplier
{
    public static int[,] BlockMultiplyMatrices(int[,] matrixA,
int[,] matrixB, int block)
    {
        int rowsOfA = matrixA.GetLength(0);
        int colsOfA = matrixA.GetLength(1);
        int colsOfB = matrixB.GetLength(1);
        int[,] resultMatrix = new int[rowsOfA, colsOfB];

        for (int iBlock = 0; iBlock < rowsOfA; iBlock += block)
        {
            for (int jBlock = 0; jBlock < colsOfB; jBlock +=
block)
            {
                for (int kBlock = 0; kBlock < colsOfA; kBlock +=
block)
                {
                    int maxI = Math.Min(iBlock + block, row-
sOfA);
                    int maxJ = Math.Min(jBlock + block,
colsOfB);
                    int maxK = Math.Min(kBlock + block,
colsOfA);

                    for (int i = iBlock; i < maxI; i++)
                    {
                        for (int j = jBlock; j < maxJ; j++)
                        {
                            int cellSum = 0;
                            for (int k = kBlock; k < maxK; k++)
                            {
                                cellSum += matrixA[i, k] * ma-
trixB[k, j];
                            }
                            resultMatrix[i, j] += cellSum;
```

```
                        }
                    }
                }
            }
        }
        return resultMatrix;
    }
    public static void DisplayMatrix(int[,] matrix)
    {
        for (int i = 0; i < matrix.GetLength(0); i++)
        {
            for (int j = 0; j < matrix.GetLength(1); j++)
            {
                Console.Write(matrix[i, j] + "\t");
            }
            Console.WriteLine();
        }
    }
    static void Main()
    {
        Random randomizer = new Random();

        Console.Write("Введите размер матриц: ");
        int size = int.Parse(Console.ReadLine());
        int[,] firstMatrix = new int[size, size];
        int[,] secondMatrix = new int[size, size];
        for (int i = 0; i < size; i++)
        {
            for (int j = 0; j < size; j++)
            {
                firstMatrix[i, j] = randomizer.Next(-10, 10);
                secondMatrix[i, j] = randomizer.Next(-10, 10);
            }
        }
        Console.WriteLine("Первая матрица:");
        DisplayMatrix(firstMatrix);
        Console.WriteLine("\nВторая матрица:");
        DisplayMatrix(secondMatrix);
        int blockSize = size > 3 ? 3 : size;
        int[,] multipliedMatrix = BlockMultiplyMatri-
ces(firstMatrix, secondMatrix, blockSize);
        Console.WriteLine("\nПроизведение:");
        DisplayMatrix(multipliedMatrix);
    }
}
```

1.4. Описание алгоритма

Блочное умножение матриц представляет собой высокоэффективный метод, который оптимизирует процесс умножения матриц путём разбиения исходных матриц на более мелкие блоки. Эти блоки обрабатываются как отдельные единицы, а затем результаты комбинируются для получения окончательной матрицы. Такой подход значительно улучшает использование кэш-памяти процессора, что может существенно ускорить вычисления, особенно при работе с большими матрицами.

Ключевым аспектом данного метода является четкая последовательность действий:

1. Деление матриц: Исходные матрицы делятся на подматрицы или блоки заранее заданного размера. Важно выбрать оптимальный размер блоков, чтобы максимально эффективно использовать кэш-память.
2. Умножение блоков: Блоки умножаются друг с другом в соответствии с правилами матричного умножения. Этот процесс может быть реализован как параллельно, так и последовательно.
3. Суммирование результатов: после выполнения умножения блоков результаты суммируются, формируя итоговую матрицу произведения.

Центральным элементом блочного умножения является выбор размера блока, который оказывает значительное влияние на скорость вычислений. С одной стороны, меньшие блоки способствуют более эффективному использованию кэш-памяти, с другой — слишком маленькие блоки могут привести к увеличению накладных расходов на управление большим количеством операций умножения. Таким образом, оптимизация размера блока является критически важной для достижения максимальной производительности данного метода.

Алгоритм создания кода.

Для начала мы создадим и объявим класс для выполнения блочного умножения матриц «class BlockMatrixMultiplier». Класс содержит статиче-

ские методы, что позволяет вызывать их без создания экземпляра класса: «BlockMultiplyMatrices». После мы запрашиваем параметры:

– int[,] matrixA: первая матрица для умножения.

– int[,] matrixB: вторая матрица для умножения.

– int block: размер блока, который будет использоваться для блочного умножения.

Далее следует получение размеров матриц: rowsOfA, colsOfA, colsOfB. После проверка на совместимость матриц для умножения (количество столбцов первой матрицы должно совпадать с количеством строк второй). Инициализация результирующей матрицы resultMatrix. Вложенные циклы для обработки блоков:

– Внешние циклы по блокам (разделяют матрицы на подматрицы).

– Внутренний цикл для вычисления значений в результирующей матрице путем суммирования произведений соответствующих элементов.

– Возвращаемое значение: Возвращает результирующую матрицу.

Далее рассмотрим статический метод «DisplayMatrix», его параметры: «int[,] matrix», это матрица, которую нужно отобразить на консоли. После происходит действие - вложенные циклы для перебора всех элементов матрицы. Вывод элементов на консоль с табуляцией для лучшего форматирования. Также в коде используется метод Main. Точка входа программы, где происходит выполнение всех действий. Создаём экземпляр Random для генерации случайных чисел: «Random random = new Random();» Запрашиваем у пользователя размера квадратных матриц. Далее инициализируем двух квадратные матрицы (firstMatrix и secondMatrix) и заполняем их случайными значениями в диапазоне от -10 до 10.

Далее мы производим вывод на консоль обеих матриц с помощью метода «DisplayMatrix». Определяем размер блока для блочного умножения. Делаем вызов метода «BlockMultiplyMatrices» для получения произведения двух матриц. Конечный вывод результирующей матрицы на консоль.

1.5. Вывод программы

```
Введите размер матриц: 10
Первая матрица:
1       9       -5      9       -1      -6      7       4       3       2
-10     -5      3       1       6       -3      9       -3      2       -2
-5      -3      -8      -8      -1      6       9       -2      9       -10
-9      -2      5       3       -2      -5      -9      -10     -3      -4
9       3       2       -7      -8      -4      -10     -2      -5      -4
5       -7      4       -8      -8      -2      -7      -6      -8      -1
7       -6      -9      1       3       -9      7       -1      5       -9
-10     2       -7      8       -10     -3      6       -7      1       9
1       -7      9       8       -5      -3      0       1       -6      3
-7      5       -7      -5      -9      3       2       7       -7      -9

Вторая матрица:
-6      3       3       -6      -8      2       6       8       0       -4
5       -1      -9      -7      -2      -8      0       -4      -10     0
3       -9      -2      2       -10     -6      -7      5       0       -7
6       -9      -9      -4      -2      2       -8      -4      -7      -6
9       6       -7      5       1       -8      -5      -1      -4      -9
5       -9      -7      6       -9      6       5       -8      1       2
7       7       -8      6       1       -9      -5      -6      2       -2
-4      -9      0       -5      -4      3       4       7       -4      2
-4      1       -4      8       -10     0       -10     0       -7      1
-4      -3      -2      -7      2       -2      -1      4       -10     7

Произведение:
52      16      -172    -124    24      -105    -107    -46     -198    -15
164     100     -97     208     88      -148    -209    -114    52      -83
39      192     -23     304     -6      23      -8      -191    159     62
39      -28     95      26      116     19      -89     -41     85      -42
-193    12      251     -169    -16     100     212     168     110     39
-172    28      302     -62     55      125     188     140     232     55
-42     288     68      89      71      -8      -43     15      105     -90
22      5       -85     -52     189     -9      -117    -170    -96     163
-18     -170    50      -88     -16     39      -50     97      39      -59
0       -28     65      -30     116     89      204     -105    139     134
Для продолжения нажмите любую клавишу . . .
```

Рисунок 1 – Результат работы алгоритма

В конце алгоритма выводятся сгенерированные матрицы, а также результат перемножения алгоритмом блочного умножения матриц.

2. Определение функциональной зависимости

Для вычисления того, как размер блоков влияет на время умножения матриц, мной был осуществлён ряд экспериментов, в процессе которых я анализировала время, необходимое для умножения матриц размером 2024×2024, в зависимости от размера блока.

Таблица 1 – Результаты эксперимента

размер (байт)	время умножения (с)
8	1,09
16	1,91
32	3,38
64	5,82
128	10,55

Я также представила данные изучения в виде диаграммы, что позволило наглядно продемонстрировать, как изменяется продолжительность операции умножения матрицы по мере увеличения размера ее блоков.

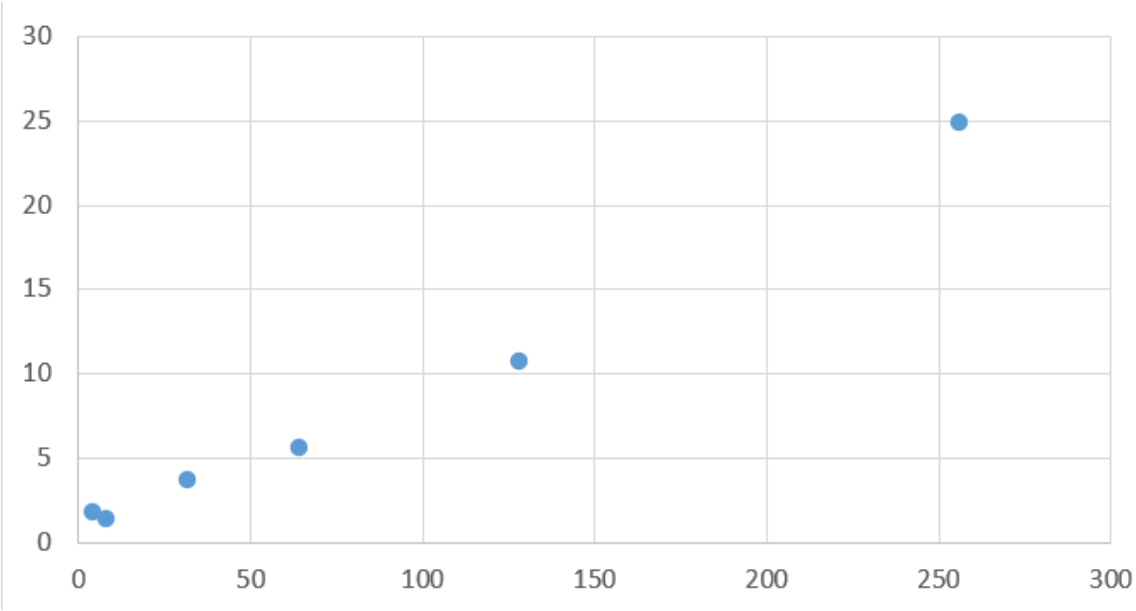

Рисунок 2 - Функциональная зависимость времени умножения матриц от размера блока

На графике видно, что время выполнения функции растёт экспоненциально с увеличением размера матриц. Также на графике видна точка минимума с размеров 8 байт с временем умножения 1, 34с для данной степенной функции.

Заключение

Метод блочного умножения матриц является подходом, который усиливает вычислительную продуктивность за счёт разделения крупных матриц на подматрицы меньшего размера. Такой процесс способствует более продуктивному взаимодействию с кэш-памятью системы.

Проведённое исследование выявило несколько ключевых аспектов:

1. Оптимизация времени выполнения вычислений за счёт использования определенного размера блоков.
2. Сокращение потребностей в оперативной памяти при использовании блоков идеального размера, что облегчает процесс обработки данных.
3. Возможность применения метода блочного умножения в широком спектре операций, включая научные исследования и обработку данных.

Таким образом, исследование показывает важность правильно выбранной стратегии разделения матриц на блоки для эффективного проведения вычислительных операций во многих областях применения.

Список использованных источников

1. Марков В.Н. Алгоритмы и структуры данных: учеб. пособие/ В.Н. Марков. – Краснодар: Изд. ФГБОУ ВО «КубГТУ», 2022. – 207 с.

2. Павловская Т.А. C#. Программирование на языке высокого уровня: Учебник для вузов. – СПб.: Питер, 2014. – 432 с.: ил.

3. Белов В.В., Чистякова В.И. Алгоритмы и структуры данных [Электронный ресурс]: учебник. – М.: КУРС: ИНФРА-М: 2020. – 240 с. URL: https://new.znanium.com/read?pid=1057212 (дата обращения:14.02.2025).

4. Колдаев В.Д. Структуры и алгоритмы обработки данных [Электронный ресурс]: учеб. пособие. – М.: РИОР: ИНФРА-М, 2014. – 296 с. URL: https://new.znanium.com/read?pid=418290 (дата обращения: 14.02.2025).

Приложение

Проверка на антиплагиат